Walkthrough: RLS Test Case 03 — Cross-Tenant Access

Test case 03 means: a user who belongs to tenant T1 must not read or mutate a protected resource owned by tenant T2 through any supported access path. Run only against local/staging with synthetic data and actual application-facing database roles. Do not run as a database owner, service role, or migration role: those can bypass the control being tested.

Preconditions

- Confirm the actual table, tenant column, owner column, policies, API/RPC path, and fixture cleanup procedure. The example names below are placeholders: `app.student_records`, `tenant_id`, `owner_id`, `private.has_capability`.
- Create UUID fixtures: `student_a` and `staff_t1` in T1; `student_c` in T2; a private record `record_t2` owned by student_c/T2.
- Start a transaction for database tests; use `SET LOCAL` so identity changes do not leak beyond it.
- Verify the policy references the same tenant/membership authority the application uses. Do not use an arbitrary client-supplied `tenant_id` as proof.

Database policy test (template)

```
begin;

-- Replace IDs and relation/columns with confirmed Semester schema.
set local role authenticated;
set local "request.jwt.claims" =
  '{"sub":"11111111-1111-1111-1111-111111111111","role":"authenticated"}';

-- Expected: zero rows. A zero-row result can be the correct RLS denial behavior.
select id, tenant_id, owner_id
from app.student_records
where id = '33333333-3333-3333-3333-333333333333'; -- record_t2

-- Expected: zero rows updated. Check the target row after rollback using an authori
update app.student_records
set display_name = 'should-not-write'
where id = '33333333-3333-3333-3333-333333333333';

-- Expected: insert rejected or zero resulting unauthorized row, depending on approv
insert into app.student_records (id, tenant_id, owner_id, display_name)
values ('44444444-4444-4444-4444-444444444444',
        '22222222-2222-2222-2222-222222222222', -- T2
        '11111111-1111-1111-1111-111111111111', -- student A
        'spoofed cross-tenant row');

rollback;
```

Supabase documents this approach: set the application-facing role and JWT claim context when diagnosing RLS behavior. Adapt the claims syntax to the installed Supabase/PostgREST versions and your actual policy helpers. [web:45]

API/RPC/storage/realtime test

1. Authenticate as T1 student A using a normal test login; retain no privileged backend key.
2. Request `record_t2` through every supported mechanism: normal API endpoint, direct RPC, export endpoint, search/list filter, attachment download, cached/derived view, and realtime subscription if applicable.
3. Attempt an update/delete and an insert that names T2 or changes tenant/owner scope.
4. Repeat as scoped T1 staff and as an intended T2-authorized user to prove the test can distinguish deny from a broken service.
5. Inspect returned rows/body, HTTP status, database side effects, storage object metadata, emitted events, logs, and audit events. Do not rely solely on a 403: zero rows can be correct for an RLS-filtered SELECT.
6. Roll back/clean synthetic fixtures and attach sanitized evidence.

Expected evidence

Record test ID `AC-03`, environment, commit/migration, policy definition hash, actor/tenant fixture IDs, target tenant/resource, request/SQL stripped of secrets, expected and actual result, affected-row count, side-effect check, log/audit correlation ID, tester, timestamp, and reviewer. Test must PASS for API, direct RPC, database policy, storage, derived/export, and realtime paths that exist. A single UI test is insufficient.

When it fails

Contain the smallest affected path; preserve evidence; find all equivalent tables/views/functions/cache/export jobs; review grants plus policies plus function ownership; fix authoritative server/database enforcement; add regression tests; retest both T1 denial and T2 authorized behavior. Treat confirmed cross-tenant student data disclosure as an incident and release blocker.